

COMP9414 Artificial Intelligence

Assignment 2: Collaborative Multi-Agent Reinforcement Learning

Term 2, 2026

Due: Friday, 31 July 2026, 5:00 PM AEST (Week 9)

Worth: 21 marks (21% of final grade)

1 Introduction

Autonomous multi-robot systems are increasingly deployed in real-world environments that require persistent, coordinated, and efficient operation. Household cleaning robots, warehouse sweepers, and industrial maintenance units all exemplify domains in which multiple agents must jointly explore, cover, and service a shared space while avoiding interference and ensuring long-term operational viability. Designing control policies for such systems is challenging due to the inherent multi-agent coordination problem, the non-stationarity introduced by simultaneously learning agents, and the long-horizon planning required to complete tasks [3] such as resource collections or dirt cleanings.

Reinforcement learning (RL) offers a promising framework for learning such behaviors directly from interaction. However, extending RL to multi-agent settings introduces additional complexity: agents must learn not only to optimize their own behavior but also to anticipate and adapt to the evolving behaviors of others [1]. Among modern RL algorithms, Proximal Policy Optimization (PPO) has emerged as a robust and stable method for continuous and discrete control. Its multi-agent variants, such as Independent PPO (IPPO) and Multi-Agent PPO with a centralized critic (MAPPO) [10], provide contrasting approaches to handling coordination and credit assignment in cooperative tasks.

This assignment focuses on the design and implementation of a simulated multi-agent PPO system for collaborative autonomous robotic cleaning using a grid-world environment. In this scenario, multiple robots must cooperatively clean a discretized environment containing obstacles, avoid collisions to fragile objects and other robots, and navigate inside the working area. These characteristics make the task an ideal testbed for studying the strengths and limitations of decentralized versus centralized-critic learning approaches.

Through this project, students will engage with the full pipeline of multi-agent reinforcement learning: formalizing the environment as a Markov Decision Problem, formulating state, action, and reward representations, building up PPO modules, implementing the baseline algorithms IPPO and MAPPO, conducting controlled experiments, evaluating models' performance, and analyzing emergent behaviors. The assignment emphasizes not only technical implementation but also critical evaluation, requiring students to interpret learning dynamics, diagnose coordination failures, enhance task efficiency, and reflect on the broader implications for multi-robot systems.

2 Learning Objectives

This assignment aims to cover topics on Neural Networks (Week 4) and Reinforcement Learning (Week 5). Students will use the architecture of the actor-critic networks implemented in multi-agent Proximal Policy Optimization (PPO). By completing this assignment, students will be able to:

1. Experiment with Neural Networks (Multi-Layer Perceptron) acting as actor and critic networks.
2. Implement a multi-agent Proximal Policy Optimization (PPO) as a class of reinforcement learning (Independent-PPO and MAPPO) for discrete agents learning and control.
3. Compare and analyze the agent’s learning behavior under different training scenarios (centralized vs decentralized), and discuss the advantages and drawbacks for each scenario.
4. Comprehensively evaluate agents’ performance: learning performance, task efficiency, co-ordination quality, and behavioural diversity.

3 Background

3.1 Proximal Policy Optimization (PPO) Reinforcement Learning

Proximal Policy Optimization (PPO) [7] is a modern policy-gradient reinforcement learning algorithm designed to provide a stable and computationally efficient alternative to earlier policy optimization methods. PPO belongs to the family of on-policy actor-critic algorithms, where a policy network (the actor) is optimized using gradient ascent, and a value function network (the critic) provides baseline estimates to reduce variance in the policy gradient. The central challenge addressed by PPO is how to update a policy in a way that improves performance while avoiding excessively large parameter shifts that can destabilize learning.

Traditional policy-gradient methods [8] optimize the expected advantage-weighted likelihood ratio

$$\mathbb{E} \left[\rho_t(\theta) \hat{A}_t \right], \quad \rho_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)},$$

where $\rho_t(\theta)$ measures how much the updated policy deviates from the behavior policy used to collect data, and $\hat{A}_t$ measures the advantage estimate. Large deviations can lead to unstable updates, especially in high-dimensional or continuous action spaces. Trust Region Policy Optimization (TRPO) [5] addressed this issue by enforcing a hard constraint on policy divergence, but at the cost of requiring second-order optimization and complex implementation.

PPO simplifies this approach by introducing a *clipped surrogate objective* that implicitly restricts policy updates without requiring second-order methods. The PPO objective is defined as

$$L_{\text{clip}}(\theta) = \mathbb{E} \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

where the clipping parameter ϵ defines a trust region around the old policy. If the policy update attempts to push $\rho_t(\theta)$ outside this region, the clipped term prevents further improvement in the objective. This mechanism ensures that policy updates remain “proximal” to the previous policy, providing stability comparable to TRPO while retaining the simplicity of first-order gradient methods.

PPO is typically implemented with an actor-critic architecture. The critic estimates the value function $V_\phi(o_t)$ and is trained using a squared-error loss

$$L_{\text{value}} = (V_\phi(o_t) - R_t)^2,$$

where R_t is a bootstrapped return estimate. Generalized Advantage Estimation (GAE) [6] is commonly used to compute $\hat{A}_t$, balancing bias and variance through the parameter λ . An entropy bonus is added to the loss function to encourage exploration and prevent premature convergence to deterministic policies.

A notable feature of PPO is its use of multiple epochs of minibatch optimization over the same set of on-policy trajectories. Although PPO is formally an on-policy algorithm, the clipped objective allows limited reuse of data without violating theoretical assumptions, significantly improving sample efficiency compared to earlier policy-gradient methods.

Due to its robustness, ease of implementation, and strong empirical performance across a wide range of continuous and discrete control tasks, PPO has become one of the most widely adopted reinforcement learning algorithms. It serves as the foundation for many multi-agent extensions, including Independent Proximal Policy Optimization (IPPO) and Multi-Agent Proximal Policy Optimization (MAPPO) [10], which adapt PPO’s core principles to decentralized and centralized multi-agent learning settings.

3.2 Independent Proximal Policy Optimization (IPPO)

Independent Proximal Policy Optimization (IPPO) is a foundational baseline in multi-agent reinforcement learning (MARL), extending the single-agent PPO algorithm to environments with multiple decision-making entities. In IPPO, each agent maintains its own policy π_{θ_i} and value function V_{ϕ_i} , and learns independently from its own local observations and rewards. This design preserves the simplicity and scalability of single-agent PPO while enabling decentralized execution. However, it also introduces a key challenge: because each agent updates its policy during training, the environment becomes *non-stationary* from the perspective of every other agent. IPPO relies on PPO’s clipped surrogate objective to mitigate instability arising from this non-stationarity, but it does not explicitly model coordination or joint value structure.

The IPPO training pipeline consists of four phases: (1) initialization of actor and critic networks, (2) decentralized data collection, (3) independent advantage estimation, and (4) independent PPO optimization. The following subsections describe how each phase operates and how they relate to the pseudocodes provided.

Actor and Critic Network Initialization

The first phase of IPPO is the initialization of Neural Networks (Algorithm 1) as the backbone for the actor and critic networks. The actor determines the policy (action probabilities) by approximating the optimal policy $\pi_\theta(a \mid o)$ to dictate the agent’s behavior, while the critic estimates state values (expected rewards) by estimating the value function $V_\phi(o_i)$ to compute the advantage $\hat{A}_t$, for stabilizing actor updates to evaluate those actions. Both networks share local observations during training.

Decentralized Data Collection

The second phase (Algorithm 2) highlights the defining characteristic of IPPO: each agent interacts with the environment using only its own local observation o_i , without access to global state information or other agents’ observations. At each timestep, agent i computes an action distribution $P(a_i) = \pi_{\theta_i}(a_i \mid o_i)$, samples an action, and evaluates its current state value $v_i =$

$V_{\phi_i}(o_i)$. The environment executes the joint action $\mathbf{a} = \{a_1, \dots, a_N\}$ and returns individual rewards and next observations.

Each agent stores its own transition tuple $(o_i, a_i, r_i, v_i, P(a_i))$ in a private buffer $\mathcal{B}_i$. No shared critic or centralized information is used during this phase. This strict decentralization makes IPPO computationally efficient and easy to scale, but it also means that each agent must learn in the presence of other agents whose policies are simultaneously changing.

Independent Advantage Estimation

The third phase (Algorithm 3) computes the Generalized Advantage Estimate (GAE) for each agent independently. For agent i , the temporal-difference error is defined as

$$\delta_{i,t} = r_{i,t} + \gamma v_{i,t+1} - v_{i,t},$$

and is used to recursively compute the advantage estimate

$$\hat{A}_{i,t} = \delta_{i,t} + \gamma \lambda \hat{A}_{i,t+1}.$$

The return target for the critic is then given by

$$R_{i,t} = \hat{A}_{i,t} + v_{i,t}.$$

Because advantage estimation is performed independently for each agent, IPPO does not incorporate joint reward structures or cross-agent credit assignment. Coordination must therefore emerge implicitly from the environment dynamics and reward design.

Independent PPO Optimization

The final phase (Algorithm 4) performs PPO updates separately for each agent. For each mini-batch, the agent computes the probability ratio

$$\rho_t(\theta_i) = \frac{P_{\text{new}}(a_i)}{P_{\text{old}}(a_i)},$$

which measures the deviation between the updated policy and the behavior policy used during data collection. The clipped surrogate objective

$$L_{\text{clip}} = -\min\left(\rho_t(\theta_i)\hat{A}_i, \text{clip}(\rho_t(\theta_i), 1 - \epsilon, 1 + \epsilon)\hat{A}_i\right)$$

prevents overly large policy updates, stabilizing learning in the presence of non-stationarity. The critic is trained using a squared-error loss between predicted values and the GAE-based return target, while an entropy bonus encourages exploration.

Because each agent optimizes its actor and critic independently, IPPO maintains full decentralization throughout training. This makes it a strong and scalable baseline, but also limits its ability to capture coordinated strategies in cooperative tasks. These limitations motivate the development of centralized-critic methods such as MAPPO, which explicitly model joint value structure to improve coordination and stability.

3.3 Multi-Agent Proximal Policy Optimization (MAPPO)

Multi-Agent Proximal Policy Optimization (MAPPO) is a centralized-critic extension of the Proximal Policy Optimization (PPO) algorithm designed for cooperative multi-agent reinforcement learning (MARL). MAPPO follows the *centralized training with decentralized execution*

(CTDE) paradigm [2], in which each agent maintains its own decentralized policy for action selection, while a centralized value function is used during training to stabilize learning and mitigate the non-stationarity inherent in multi-agent environments. This architecture allows MAPPO to preserve the scalability and independence of decentralized policies at execution time, while leveraging global information during training to improve coordination, credit assignment, and sample efficiency.

In contrast to independent learning methods such as IPPO, where each agent learns solely from its own local observations and rewards, MAPPO introduces a centralized critic $V_\phi(s)$ that conditions on the global state s . This critic provides a consistent estimate of the joint value landscape, reducing variance in advantage estimation and enabling coordinated policy updates across agents. The actor networks remain decentralized, mapping each agent’s local observation o_i to an action distribution $\pi_{\theta_i}(a_i | o_i)$, ensuring that execution does not require access to global information.

The MAPPO training pipeline consists of four tightly coupled phases: (1) constructing actor and critic networks, (2) decentralized data collection, (3) centralized advantage estimation, and (4) centralized PPO optimization. The pseudocode in Algorithms 5–8 illustrates how these phases are implemented in practice.

Actor and Critic Networks Initialization

The first phase of MAPPO is the initialization of Neural Networks as the backbone for the actor and critic networks (Algorithm 5). The actor determines the policy (action probabilities) by approximating the optimal policy $\pi_\theta(a | o)$ to dictate the agent’s behavior, while the critic estimates state values (expected rewards) by estimating the value function $V_\phi(o_i)$ to compute the advantage $\hat{A}_t$, for stabilizing actor updates to evaluate those actions. Multi-Agent Proximal Policy Optimization (MAPPO) uses an actor-critic architecture with a Centralized Training with Decentralized Execution (CTDE) paradigm. During training, a shared actor network manages each agent’s policy, while a centralized critic network evaluates joint performance to stabilize learning and resolve multi-agent credit assignment.

Decentralized Data Collection

The second phase (Algorithm 6) captures the CTDE structure of MAPPO. Each agent selects actions using only its local observation o_i , ensuring that execution remains decentralized. However, unlike IPPO, the value estimate $v_i = V_{\phi_i}(s)$ is computed using the *global state* s , which is available only during training. This centralized value evaluation provides a stable learning signal that is consistent across agents, even as their policies evolve.

At each timestep, the environment executes the joint action $\mathbf{a} = \{a_1, \dots, a_N\}$ and returns individual rewards, the next global state s' , and updated local observations. Each agent stores its transition tuple $(o_i, s, a_i, r_i, v_i, P(a_i))$ in its trajectory buffer $\mathcal{B}_i$. The inclusion of the global state in each transition is essential for centralized advantage estimation and critic training.

Centralized Advantage Estimation

The third phase (Algorithm 7) computes the Generalized Advantage Estimate (GAE) for each agent using the centralized critic. For agent i , the temporal-difference error is defined as

$$\delta_{i,t} = r_{i,t} + \gamma V_{\phi_i}(s_{t+1}) - V_{\phi_i}(s_t),$$

where both value terms depend on the global state. This centralized TD error provides a consistent and low-variance estimate of the joint return, addressing the non-stationarity that arises when multiple agents learn simultaneously.

The advantage estimate is computed recursively as

$$\hat{A}_{i,t} = \delta_{i,t} + \gamma \lambda \hat{A}_{i,t+1},$$

and the return target for the critic is given by

$$R_{i,t} = \hat{A}_{i,t} + V_{\phi_i}(s_t).$$

Because all agents use the same global state information for value estimation, MAPPO produces advantage signals that are aligned across agents, facilitating coordinated policy improvement.

Centralized PPO Optimization

The final phase (Algorithm 8) performs PPO optimization using the centralized critic and decentralized actors. For each agent, the probability ratio

$$\rho_t(\theta_i) = \frac{\pi_{\theta_i}(a_i | o_i)}{\pi_{\theta_i^{\text{old}}}(a_i | o_i)}$$

is computed using only local observations, preserving decentralized execution constraints. The clipped surrogate objective

$$L_{\text{clip}} = -\min \left(\rho_t(\theta_i) \hat{A}_i, \text{clip}(\rho_t(\theta_i), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right)$$

ensures stable policy updates, while the critic loss

$$L_{\text{value}} = (V_{\phi_i}(s) - R_i)^2$$

trains the centralized value function using global state information. An entropy bonus encourages exploration, and the total loss combines actor, critic, and entropy terms.

Because MAPPO typically employs parameter sharing across agents in symmetric environments, the actor and critic networks can be updated using experiences from all agents, improving sample efficiency and promoting coordinated behavior. The centralized critic ensures that all agents optimize their policies with respect to a consistent estimate of the joint return, enabling MAPPO to achieve strong performance in cooperative MARL tasks.

4 Implementation Tasks

Students are required to complete 5 tasks: (1) designing a simulated environment for MARL, (2) constructing supporting modules for PPO pipeline, (3) implementing Independent Proximal Policy Optimization (IPPO) and (4) Multi-Agent Proximal Policy Optimization (MAPPO) for multi-robot cleaners simulation, and (5) performing a comprehensive evaluation. Each task comprises some required deliverables for assessment.

4.1 Task 1: Environment for Multi-Agent Reinforcement Learning (MARL)

In Task 1, students will (1) design a grid world environment to simulate Multi-Agent Reinforcement Learning (MARL) for the scenario of a collective cleaning task, (2) visualize the grid-world environment, agents, and objects following experiment specifications, (3) demonstrate the simulation using random actions to verify the per-agent states and rewards for each step, and (4) verify that some penalties, conditions for truncation and termination are properly set up.

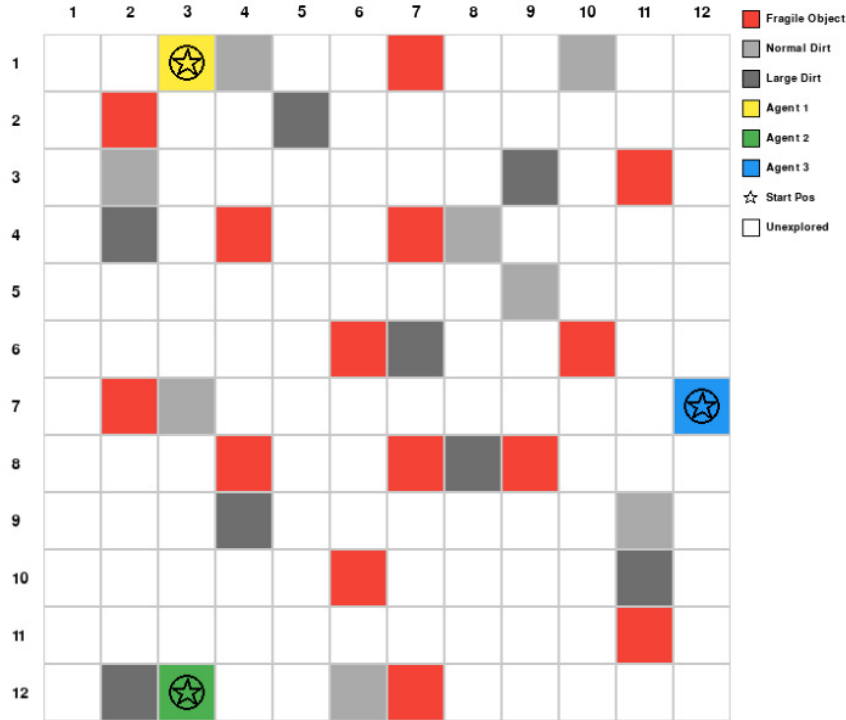


Figure 1: Depiction of the 12 x 12 Collaborative Cleaning Area Grid World. Fragile objects are shown in red, large dirt is in dark grey, small dirt is in light grey, Agent 1 is in yellow, Agent 2 is in green, Agent 3 is in blue, the start positions for each agent are marked as $\star$, and unexplored cells are in white.

Environment Specification: Students will experiment with a cooperative multi-agent Grid World in which $N = 3$ agents operate on a discrete 12×12 environment. The map contains fragile objects, dirty cells (normal dirt and large dirt), unexplored cells, and three fixed docking positions as the start position of each agent. Students must implement this environment using a standard interface (Gymnasium [9] or a similar library) to ensure compatibility with your RL agents. The environment must be self-contained and support all standard operations required for multi-agent reinforcement learning. The environment should be designed to facilitate (1) a fixed position scenario where all dirt and fragile objects are stationary (see Table 1) and (2) a random position scenario where the normal dirt and large dirt are randomly populated, with stationary fragile objects' locations. The objective is to clean all dirt (both normal and large sizes) without necessarily returning to the start position (docking location). During simulation, a newly explored cell will be assigned to the first agent arrival, hence must follow the agent's color and ID for spatial distinction. Each episode runs for a maximum of 50 time steps. If the time limit is reached before all dirt is cleaned, the episode is truncated.

Table 1: Grid World Specification

Cell Type	Coordinate (row, col)	Amount
Start Positions	Agent 1:(1, 3), Agent 2: (12, 3), Agent 3: (7, 12)	3
Fragile Objects	(2, 2), (4, 4), (6, 6), (7, 2), (8, 4), (10, 6), (1, 7), (4, 7), (8, 7), (12, 7), (3,11), (8, 9), (6, 10), (11, 11),	14
Large-size Dirt	(2, 5), (3, 9), (4, 2), (6, 7), (8, 8), (9, 4), (10, 11), (12, 2)	8
Normal-size Dirt	(1, 4), (1, 10), (3, 2), (4, 8), (5, 9), (7, 3), (9, 11), (12, 6)	8

Action Space: Agent's action is encoded into 5 distinct actions {IDLE=0, UP=1, DOWN=2, LEFT=3, RIGHT=4}. The (row, col) coordinates for the key components are specified in Figure 1. Actions map to grid movements as follows: IDLE maintains hold position (no change in row and col), UP decreases the row index (row - 1), DOWN increases the row index (row + 1), LEFT decreases the column index (col - 1), and RIGHT increases the column index (col + 1). Each agent starts from a designated docking location and must jointly explore the map, avoid fragile cells and collisions, and clean all dirty cells. A safety shield mechanism should be implemented as a soft collision and boundary-going prevention, in which each agent will execute the IDLE action to avoid collision.

Agent state representation: Each agent will have their own local observation space with a tensor of size $\{12, 12, C\}$, where C is a channel consisting of 5 main pieces of information: {the fragile-cell map, the self-position map of agent, the other-agent location, the explored-area map, and the start-position of agent}. The joint observation space will combine all the local agent observation spaces to form a global observation space (especially significant for centralized training). The robots must collectively clean all dirty cells while avoiding collisions with other agents and fragile objects. The problem is formulated as a cooperative partially observable Markov Decision Process, where each agent receives local observations and must learn a policy that contributes to the success of the team as a whole. At each time step t , agent $i \in \{1, 2, 3\}$ receives a local observation tensor $o_t^i \in \{0, 1\}^{12 \times 12 \times 5}$, which serves as the agent's state representation for decentralized control. Each observation is composed of five binary channels encoding the most relevant spatial information for decision-making. Let F_t , S_t^i , O_t^i , E_t , and D_t^i denote the fragile-cell map, the self-position map of agent i , the other-agent occupancy map, the explored-area map, and the docking-position map of agent i , respectively. Then the local observation of agent i is written as $o_t^i = [F_t, S_t^i, O_t^i, E_t, D_t^i]$.

Reward shaping: To encourage safe exploration for cleaning, the reward function will be defined as in Table 2. Each reward will be accumulated if, at the same time-step, the agent experiences more than one reward-triggering event. For instance, if an agent explores a new cell containing normal dirt, it will receive rewards for exploring the new cell (+2) and cleaning the normal dirt (+2).

Table 2: Assignment of Reward Values

Event Triggering Reward	Reward Value	Implication
A normal size dirt cleaned	+2	Keep going
A large size dirt cleaned	+4	Keep going
A new cell explored	+2	Keep going
Revisit explored cell	-1	Keep going
Idle penalty (to discourage a passive action)	-0.1	Keep going
Time step penalty (shared across team)	-0.1	Keep going
Collision with a fragile object or another robot	-20	Truncated
Going beyond the grid	-20	Truncated
Completion Bonus (CLEANED all dirty cells)	+20	Terminated

Required Deliverables for Task 1	
1.	A working Python implementation of the grid world environment.
2.	A clear visualization of the grid world using Matplotlib or a similar library.
3.	A demonstration script that runs a few random actions and prints the state and reward vector at each step for each agent.
4.	Verification that collision to fragile object and other robot truncates the episode and provides the correct penalty vector.

4.2 Task 2: Modular PPO Pipeline Construction for Multi-Robot System

In Task 2, students will develop the main supporting modules for the PPO experiment and perform sanity check/build verification testing for each module, namely,

- **PPO Actor and Critic Network:** (1) Constructing the base architecture for each network, (2) performing a sanity check, (3) validating the architecture by showing summary.
- **Rollout buffer module (Algorithm 2):** (1) Creating function for rollout data generator by connecting the actor and critic network to the MARL Environment via actor policy and value, (2) generating rollout data from the untrained PPO Actor and Critic networks, (3) saving a one-cycle rollout data (observation, action, log-probability of each action, reward vector, and values) into a numpy archive (.npz) or pickle format, and (4) printing the shape/size and value of each rollout data.
- **Advantage estimator module (Algorithm 3):** (1) Creating the sub-functions for advantage estimation, (2) performing GAE estimation using the saved rollout data, and (3) analyzing the bias and variance from the generated advantage value.
- **Optimizer module (Algorithm 4):** (1) constructing functions for computing action probabilities, new state values, probability ratio, actor loss with clip, critic loss, entropy bonus, total loss, and network update, (2) integrating all functions into optimization module, (3) use the generated rollout data for computing actor loss, critic loss, bonus entropy, and total loss, and (4) displayed all computed values.

In this task, the system is assumed to be an open-loop system - each module should be prepared to be interconnected without a feedback loop for iteration, for a one-cycle data demonstration. In building those modules, students must follow the IPPO pseudocode (Algorithm 1 - 4), where each module is formulated.

	Required Deliverables for Task 2
1.	A modular codebase and result for PPO Actor and Critic networks test.
2.	A modular codebase and result for rollout buffer module test.
3.	A modular codebase and result for the advantage estimator module test.
4.	A modular codebase and result for the optimizer module test.

Algorithm 1 IPPO Main Loop

```

1: Initialize:  $N$  agents
2: for each agent  $i \in \{1, \dots, N\}$  do
3:   Phase 1A: Initialize actor network  $\pi_{\theta_i}$  with parameters  $\theta_i$ 
4:   Phase 1B: Initialize critic network  $V_{\phi_i}$  with parameters  $\phi_i$ 
5: end for
6: for iteration = 1, 2, ..., MAX_ITERATIONS do
7:   Execute Phase 2: Decentralized Data Collection ▷ See Algorithm 2
8:   Execute Phase 3: Independent Advantage Estimation ▷ See Algorithm 3
9:   Execute Phase 4: Independent PPO Optimization ▷ See Algorithm 4
10: end for

```

Algorithm 2 IPPO Phase 2: Decentralized Data Collection

```

1: Reset environment, get initial individual observations  $\mathbf{o} = \{o_1, \dots, o_N\}$ 
2: Initialize empty trajectory buffers  $\mathcal{B}_i$  for all agents  $i \in \{1, \dots, N\}$ 
3: for  $t = 1$  to  $T_{\text{steps}}$  do
4:   for each agent  $i$  do
5:     Compute action probabilities:  $P(a_i) = \pi_{\theta_i}(a_i|o_i)$ 
6:     Sample action:  $a_i \sim P(a_i)$ 
7:     Estimate state value:  $v_i = V_{\phi_i}(o_i)$ 
8:   end for
9:   Execute joint action  $\mathbf{a} = \{a_1, \dots, a_N\}$  in the environment
10:  Receive joint rewards  $\mathbf{r} = \{r_1, \dots, r_N\}$  and next observations  $\mathbf{o}' = \{o'_1, \dots, o'_N\}$ 
11:  for each agent  $i$  do
12:    Store transition  $(o_i, a_i, r_i, v_i, P(a_i))$  in buffer  $\mathcal{B}_i$ 
13:    Update observation:  $o_i \leftarrow o'_i$ 
14:  end for
15: end for

```

Algorithm 3 IPPO Phase 3: Independent Advantage Estimation

```

1: for each agent  $i \in \{1, \dots, N\}$  do
2:   Initialize  $A_t = 0$  for terminal step
3:   for  $t = T_{\text{steps}}$  down to 1 do
4:     Extract  $r_{i,t}$ ,  $v_{i,t}$ , and  $v_{i,t+1}$  from buffer  $\mathcal{B}_i$ 
5:     Compute TD Error:  $\delta_{i,t} = r_{i,t} + \gamma v_{i,t+1} - v_{i,t}$ 
6:     Compute Generalized Advantage Estimate (GAE):
7:        $\hat{A}_{i,t} = \delta_{i,t} + \gamma \lambda \hat{A}_{i,t+1}$ 
8:     Compute Return (Target Value):  $R_{i,t} = \hat{A}_{i,t} + v_{i,t}$ 
9:     Store  $\hat{A}_{i,t}$  and  $R_{i,t}$  in  $\mathcal{B}_i$ 
10:   end for
11: end for

```

Algorithm 4 IPPO Phase 4: Independent PPO Optimization

```

1: for each agent  $i \in \{1, \dots, N\}$  do
2:   for epoch = 1 to  $K_{\text{epochs}}$  do
3:     Shuffle buffer  $\mathcal{B}_i$  and divide into mini-batches
4:     for each mini-batch do
5:       Compute new action probabilities:  $P_{\text{new}}(a_i) = \pi_{\theta_i}(a_i|o_i)$ 
6:       Compute new state values:  $v_{\text{new}} = V_{\phi_i}(o_i)$ 
7:       Actor Loss:
8:       Probability Ratio:  $\rho_t(\theta_i) = \frac{P_{\text{new}}(a_i)}{P_{\text{old}}(a_i)}$ 
9:        $L_{\text{clip}} = -\min\left(\rho_t(\theta_i)\hat{A}_i, \text{clip}(\rho_t(\theta_i), 1 - \epsilon, 1 + \epsilon)\hat{A}_i\right)$ 
10:      Critic Loss:
11:       $L_{\text{value}} = (v_{\text{new}} - R_i)^2$ 
12:      Entropy Bonus:
13:       $S_{\text{entropy}} = \text{Entropy}(\pi_{\theta_i}(o_i))$ 
14:      Total Loss:
15:       $L_{\text{total}} = \text{mean}(L_{\text{clip}}) + c_1 \cdot \text{mean}(L_{\text{value}}) - c_2 \cdot \text{mean}(S_{\text{entropy}})$ 
16:      Network Update:
17:      Update  $\theta_i$  and  $\phi_i$  by performing gradient descent on  $L_{\text{total}}$  using Adam
18:    end for
19:  end for
20:  Clear trajectory buffer  $\mathcal{B}_i$ 
21: end for

```

4.3 Task 3: Independent Proximal Policy Optimization (IPPO)

In Task 3, students will (1) design an algorithm based on Independent Proximal Policy Optimization (IPPO) with fully independent actor and critic networks, (2) conduct training using 2 modes (fixed and random dirt positions), plot the training curves, and record all policies and values, (3) evaluate performance using metrics of episode return, episode length, success rate, and safety violations, and (4) demonstrate learned model simulation using the trained weight.

Experimental setups: IPPO is a multi-agent extension of PPO where each agent learns its own policy independently, even though they operate in a shared environment. IPPO treats each agent as if it is running its own single-agent PPO algorithm, using only its local observations and shared team reward. In this design, students will implement the decentralized actor-critic network for each agent, using a Neural Network (MLP). There are 4 core stages in implementing IPPO: (1) Initialization of the actor and critic network, (2) Decentralized data collection, (3) Advantage estimation, and (4) PPO-based optimization. The pseudocodes for the IPPO are presented in Algorithm 1–4.

Modes of Experiment: Students are required to experiment using PPO streamline applied to 3 agents. The actor and critic have completely independent neural networks with no parameter sharing. The inputs are sourced from the per-agent local observation only. Students will train the model using 2 different modes: stationary and random dirt positions.

Training Protocol: IPPO trains each agent as an independent PPO learner. Every agent maintains its own policy and value function, and updates them using only its local observations and local rewards. During execution, each agent acts autonomously using its own policy network. No cross-agent communication or shared critic is required. Students are then required to:

- **Report the plot of training curves** by showing side-by-side of each mode the total loss, actor loss, critic loss, and entropy bonus per agent per episode.
- **Record** all policies and values during training and export them as a .npz (Numpy Archive) or .pkl (Pickle) file.
- **Demonstrate the execution of the learned algorithm** for testing in the same grid map (it is recommended to use live simulation with Gymnasium) and visually observe the results.

Evaluation metrics: In evaluating both experiments, students must use the same architecture and hyperparameter settings for the actor and critic networks (2-4 hidden layers, using 32 - 256 neurons per layer) to maintain fair evaluation and analysis. For each agent in both experiment modes, evaluate their performance over 200- 500 episodes, with a maximum of 50 steps per episode, using the hyperparameters specified in Table 3. Report the following metrics:

- **Average Return:** The mean cumulative rewards per episode, comparing each agent and collective team rewards.
- **Episode Length:** The mean number of steps to reach the collection of all rewards, averaged over successful episodes only. Report N/A if no episodes reached the goal. An episode is successful when all dirt is cleaned before the time limit.
- **Success Rate:** The percentage of dirt cleaned per episode. In this context, the large dirt is worth twice the normal dirt for the success rate.
- **Safety Violations:** The fraction of test episodes that were truncated by entering fragile-object cells and collision with other robots.

Table 3: multi-agent PPO hyperparameter settings (IPPO and MAPPO)

Hyperparameter	Range Value
Discount factor (γ)	0.90 - 0.98
GAE parameter (λ)	0.90 - 0.98
PPO clipping threshold (ϵ)	0.10 - 0.20
Actor learning rate	1×10^{-5} - 2×10^{-3}
Critic learning rate	1×10^{-5} - 2×10^{-3}
Rollout length	200 - 600
Minibatch size	100 - 400
Update epochs	4 - 32
Weight decay	1×10^{-6} - 1×10^{-4}

	Required Deliverables for Task 3
1.	Modular Codebase: A clean and well-commented code, showing step-by-step sub-task fulfillment.
2.	IPPO Training Results: Comparing 2 modes of training (fixed and random dirt positions) by plotting training curves per agent over training episodes of total loss, actor loss, critic loss, and entropy bonus, and saving the best model weights (checkpoint) and training metrics to .npz files.
3.	IPPO Performance Evaluation: Comparing 2 modes of training (fixed and random dirt positions) by plotting training metrics per agent over training episodes of episode return, episode length, success rate, and safety violations.
4.	IPPO Visual Demonstration: Loading the best checkpoint and executing the learned actor policy (by detaching the critic network) in simulation (Gymnasium with render mode "human").

4.4 Task 4: Multi-Agent Proximal Policy Optimization (MAPPO)

In Task 4, students will (1) design an algorithm based on Multi-Agent Proximal Policy Optimization (MAPPO) with shared actors and centralized critic, (2) conduct training using 2 modes (fixed and random dirt positions), plot the training curves, and record all policies and values, (3) evaluate performance using metrics of episode return, episode length, success rate, and safety violations, and (4) demonstrate learned model simulation using the trained weight.

Experimental setups: MAPPO is unique because it uses a centralized critic and shared policy to stabilize learning and enable coordinated multi-agent behavior, whereas IPPO trains each agent independently with only local information. MAPPO uses a centralized value function that sees the joint observations or global state, giving it a complete picture of the multi-agent system. In this experiment, students will implement the MAPPO architecture by leveraging actor-critic networks with Neural Networks. There are 4 core stages in implementing MAPPO: (1) Initialization of the actor and critic network, (2) Decentralized data collection, (3) Advantage estimation, and (4) PPO-based optimization. The pseudocodes for the MAPPO are presented in Algorithm 5–8

Modes of Experiment: Students experiment using Multi-Agent PPO streamline applied to 3 agents, where a single parameter-sharing actor is utilized with a one-hot agent-ID input that implements the per-agent policies, and a global critic that combines observations of all agents. The actor and critic have completely independent neural networks with no parameter sharing. Students train the model under 2 different modes: stationary and random dirt positions.

Training Protocol: MAPPO is a multi-agent extension of PPO that follows the *Centralized Training with Decentralized Execution (CTDE)* paradigm. This means, during training, agents

can use global information (e.g., full state observations) through a centralized critic. However, during inference, each agent must act independently using only its local observation. Students are required to:

- **Report the plot of training curves** by showing side-by-side of each mode the total loss, actor loss, critic loss, and entropy bonus per agent per episode.
- **Record** all policies and values during training and export them as a .npz (Numpy Archive) or .pkl (Pickle) file.
- **Demonstrate the execution of the learned algorithm** for testing in the same grid map (it is recommended to use live simulation with Gymnasium) and visually observe the results.

Evaluation metrics: In evaluating both experiments, students must use the same architecture and hyperparameter settings for the actor and critic networks (2-4 hidden layers, using 32 - 256 neurons per layer) to maintain fair evaluation and analysis. For each agent in both experiment modes, evaluate their performance over 200-500 episodes, with a maximum of 50 steps per episode, using the hyperparameters specified in Table 3. Report the following metrics:

- **Average Return:** The mean cumulative rewards per episode, comparing each agent and collective team rewards.
- **Episode Length:** The mean number of steps to reach the collection of all rewards, averaged over successful episodes only. Report N/A if no episodes reached the goal. An episode is successful when all dirt is cleaned before the time limit.
- **Success Rate:** The percentage of dirt cleaned per episode. In this context, the large dirt is worth twice the normal dirt for the success rate.
- **Safety Violations:** The fraction of test episodes that were truncated by entering fragile-object cells and collision with other robots.

	Required Deliverables for Task 4
1.	Modular Codebase: A clean and well-commented code, showing step-by-step sub-task fulfillment.
2.	MAPPO Training Results: Comparing 2 modes of training (fixed and random dirt positions) by plotting training curves per agent over training episodes of total loss, actor loss, critic loss, and entropy bonus, and saving the best model weights (checkpoint) and training metrics to .npz files.
3.	MAPPO Performance Evaluation: Comparing 2 modes of training (fixed and random dirt positions) by plotting training metrics per agent over training episodes of episode return, episode length, success rate, and safety violations.
4.	MAPPO Visual Demonstration: Loading the best checkpoint and executing the learned actor policy (by detaching the critic network) in simulation (Gymnasium with render mode "human").

Algorithm 5 MAPPO Main Loop

```

1: Initialize:  $N$  agents
2: for each agent  $i \in \{1, \dots, N\}$  do
3:   Phase 1A: Initialize actor network  $\pi_{\theta_i}(o_i)$  with parameters  $\theta_i$ 
4:   Phase 1B: Initialize centralized critic network  $V_{\phi_i}(s)$  with parameters  $\phi_i$ 
5: end for
6: for iteration = 1, 2, ..., MAX_ITERATIONS do
7:   Execute Phase 2: Data Collection (Decentralized Execution) ▷ See Alg 6
8:   Execute Phase 3: Centralized Advantage Estimation ▷ See Alg 7
9:   Execute Phase 4: MAPPO Optimization (Centralized Training) ▷ See Alg 8
10: end for

```

Algorithm 6 MAPPO Phase 2: Data Collection (Decentralized Execution)

```

1: Reset environment, get initial global state  $s$  and local observations  $\mathbf{o} = \{o_1, \dots, o_N\}$ 
2: Initialize empty trajectory buffers  $\mathcal{B}_i$  for all agents  $i \in \{1, \dots, N\}$ 
3: for  $t = 1$  to  $T_{\text{steps}}$  do
4:   for each agent  $i$  do
5:     Compute action probabilities using local obs:  $P(a_i) = \pi_{\theta_i}(a_i|o_i)$ 
6:     Sample action:  $a_i \sim P(a_i)$ 
7:     Estimate state value using global state:  $v_i = V_{\phi_i}(s)$ 
8:   end for
9:   Execute joint action  $\mathbf{a} = \{a_1, \dots, a_N\}$  in the environment
10:  Receive joint rewards  $\mathbf{r} = \{r_1, \dots, r_N\}$ , next global state  $s'$ , and next observations  $\mathbf{o}'$ 
11:  for each agent  $i$  do
12:    Store transition  $(o_i, s, a_i, r_i, v_i, P(a_i))$  in buffer  $\mathcal{B}_i$ 
13:    Update observation:  $o_i \leftarrow o'_i$ 
14:  end for
15:  Update global state:  $s \leftarrow s'$ 
16: end for

```

Algorithm 7 MAPPO Phase 3: Centralized Advantage Estimation

```

1: for each agent  $i \in \{1, \dots, N\}$  do
2:   Initialize  $A_t = 0$  for terminal step
3:   for  $t = T_{\text{steps}}$  down to 1 do
4:     Extract  $r_{i,t}$ , global state values  $v_{i,t}$ , and  $v_{i,t+1}$  from buffer  $\mathcal{B}_i$ 
5:     Compute TD Error based on global state evaluation:
6:        $\delta_{i,t} = r_{i,t} + \gamma v_{i,t+1} - v_{i,t}$ 
7:     Compute Generalized Advantage Estimate (GAE):
8:        $\hat{A}_{i,t} = \delta_{i,t} + \gamma \lambda \hat{A}_{i,t+1}$ 
9:     Compute Return (Target Value):  $R_{i,t} = \hat{A}_{i,t} + v_{i,t}$ 
10:    Store  $\hat{A}_{i,t}$  and  $R_{i,t}$  in  $\mathcal{B}_i$ 
11:   end for
12: end for

```

Algorithm 8 MAPPO Phase 4: Optimization (Centralized Training)

```

1: for each agent  $i \in \{1, \dots, N\}$  do
2:   for epoch = 1 to  $K_{\text{epochs}}$  do
3:     Shuffle buffer  $\mathcal{B}_i$  and divide into mini-batches
4:     for each mini-batch do
5:       Compute new action probs (Decentralized):  $P_{\text{new}}(a_i) = \pi_{\theta_i}(a_i|o_i)$ 
6:       Compute new state values (Centralized):  $v_{\text{new}} = V_{\phi_i}(s)$ 
7:       Actor Loss (Decentralized execution bound):
8:         Probability Ratio:  $\rho_t(\theta_i) = \frac{P_{\text{new}}(a_i)}{P_{\text{old}}(a_i)}$ 
9:          $L_{\text{clip}} = -\min\left(\rho_t(\theta_i)\hat{A}_i, \text{clip}(\rho_t(\theta_i), 1 - \epsilon, 1 + \epsilon)\hat{A}_i\right)$ 
10:      Critic Loss (Centralized training bound):
11:         $L_{\text{value}} = (v_{\text{new}} - R_i)^2$ 
12:      Entropy Bonus:
13:         $S_{\text{entropy}} = \text{Entropy}(\pi_{\theta_i}(o_i))$ 
14:      Total Loss:
15:         $L_{\text{total}} = \text{mean}(L_{\text{clip}}) + c_1 \cdot \text{mean}(L_{\text{value}}) - c_2 \cdot \text{mean}(S_{\text{entropy}})$ 
16:      Network Update:
17:        Update  $\theta_i$  and  $\phi_i$  by performing gradient descent on  $L_{\text{total}}$ 
18:    end for
19:  end for
20:  Clear trajectory buffer  $\mathcal{B}_i$ 
21: end for

```

4.5 Task 5: Comprehensive Evaluation

For this assignment, the collaborative multi-robot cleaning task will be evaluated on 4 core aspects: (1) learning performance, (2) task efficiency and coordination quality, (3) agents' contribution towards teamwork success, and (4) behavioral diversity. Students are required to compare all 4 evaluation aspects for **IPPO** and **MAPPO**.

1. Learning performance: This part will evaluate and compare IPPO and MAPPO performances on unseen data in random dirt placement scenarios. Students are required to:

- Load the actor's trained weights (by detaching the critic network) and evaluate the performance on the grid world environment. Utilize a different random seed than the one used during the training. In this test, use 400 episodes for each mode.
- Plot the success rate and collected rewards over the length of test episodes, and print the value of the average violation rate and average episode length for each mode.
- Generate a box plot comparing both mode data distributions, comparing success rate and return per agent per mode.

2. Task Efficiency and Coordination: This part will evaluate task efficiency and coordination quality by measuring redundancy or overlapping works done by all agents as a team:

- Global Redundant Coverage Ratio (*RCR*) as the ratio of steps the agents spent traversing cells that have already been explored by the agent or other agents. Global RCR summarizes the entire map with one number. Lower redundancy implies the agents have successfully learned coordination and efficient teamwork to partition the workload in the 12×12 grid geometry.

$$\text{Global RCR} = \frac{\sum_{x,y} \text{revisit_map}(x,y)}{\max\left(1, \sum_{x,y} \text{visit_map}(x,y)\right)}$$

The $\text{visit_map}(x,y)$ counts how many times cell (x,y) was visited during test or inference. The $\text{revisit_map}(x,y)$ counts how many of those visits were revisits, meaning the cell had already been explored before the agent came back to it. In this formulation, the numerator is the total number of revisited cell events across the entire grid, and the denominator is the total number of visits across the entire grid. That makes Global RCR a team-level redundancy rate, as $\text{Global RCR} \approx$ fraction of visits that were redundant.

- RCR Map as a heatmap visualization showing visitation redundancy in each cell. RCR map shows redundancy cell by cell calculated using formula as follows,

$$\text{RCRMap}(x,y) = \begin{cases} \frac{\text{revisit_map}(x,y)}{\text{visit_map}(x,y)}, & \text{if } \text{visit_map}(x,y) > 0, \\ 0, & \text{otherwise} \end{cases}$$

3. Agents Contribution: The advantage estimate for each agent per episode can be compared to measure each agent's contribution toward the team's success.

$$A_i = r + \gamma V(s') - V(s),$$

where $V(s)$ is the value function. Aggregating these advantages episode during inference on random scenarios provides insight into each agent's learning contribution.

4. Behavioural Diversity: The Jensen–Shannon Divergence (JSD) [4] is a symmetric and bounded measure of dissimilarity between two probability distributions. It is widely used in multi-agent reinforcement learning (MARL) to quantify behavioral differences between agents' policies under identical observations. Let P and Q be two probability distributions over the same action space $\mathcal{A}$. Define the mixture distribution $(M) = \frac{1}{2}(P + Q)$.

The Jensen–Shannon Divergence between P and Q is defined as:

$$JS(P\|Q) = \frac{1}{2}KL(P\|M) + \frac{1}{2}KL(Q\|M),$$

where $KL(\cdot\|\cdot)$ denotes the Kullback–Leibler divergence:

$$KL(P\|M) = \sum_{a \in \mathcal{A}} P(a) \log \frac{P(a)}{M(a)}.$$

		Required Deliverables for Task 5
Each Comparing IPPO & MAPPO for fixed & random settings	1.	Analytical data for generalization: both modes comparison for success rate and rewards (box plot) and average violation rate and episode length.
	2.	Analytical data for Global Redundant Coverage Ratio (RCR) and redundancy heatmap in the grid world (RCR map).
	3.	Analytical data for agents' contribution.
	4.	Analytical plot of Jensen-Shannon Divergence (JSD) in the form of a pairwise divergence matrix built from 3 agents.

5 Assessment

The assignment is marked out of 21, with 9 marks allocated to implementation and 12 marks to conceptual understanding and analysis. A detailed breakdown is provided in Table 4. Note that implementation marks are awarded for correctly functioning code, while discussion marks are awarded for the quality and depth of your understanding during the oral discussion session.

Table 4: Assessment Criteria

Category	Component	Marks
Implementation (9)	Task 1: Environment for MARL simulation	1
	Task 2: Modular PPO Build and Sanity Check	2
	Task 3: Independent Proximal Policy Optimization (IPPO)	2
	Task 4: Multi-Agent Proximal Policy Optimization (MAPPO)	2
	Task 5: Comprehensive Evaluation	2
Understanding & Discussion (12)	Multi-Agent Reinforcement Learning concepts:	4
	Policy gradient, actor-critic, PPO streamline	
	Neural Network concepts:	4
	Architecture, hyperparameters, loss function	
	Experimental analysis on IPPO and MAPPO:	4
	Learning performance, task efficiency, emergent behaviors	
TOTAL		21

Oral Discussion Sessions: All students must attend a mandatory individual oral discussion with their tutor. During this session, students will be expected to demonstrate their working system and answer conceptual and practical questions about multi-agent RL, PPO, IPPO, and MAPPO. The discussion assesses students' genuine understanding of the code they have submitted. Failure to participate in the oral discussion will result in a mark of zero for the entire assignment. Scheduling of these sessions will be handled through Moodle in the weeks following the submission deadline.

6 Submission and Late Penalties

6.1 Submission Instructions

Submit a zip archive named `zID assignment2.zip` (where `zID` is student ID) containing:

1. Your Jupyter notebook (.ipynb) with all implementation code and results (plots and tables) for Tasks 1 to 5.
2. Saved artefacts so that tutors can reproduce your results without retraining.

Your notebook must be well-structured and easy to follow. Use Markdown cells to separate the tasks clearly. All code should be appropriately commented, variable names should be descriptive, and all figures must be rendered and visible before submission.

Submission Platform: Submit electronically via the Moodle assignment portal. Multiple submissions are allowed until the deadline; only the latest version will be marked. After submitting, it is your responsibility to verify that the uploaded file is correct and complete. Take a screenshot of the submission confirmation for your records.

Self-contained Execution: Your notebook should be runnable from start to finish without errors. We recommend using a fresh virtual environment to test your notebook before final submission. If any specific library versions are required, please include a requirements cell at the top of your notebook.

6.2 Late Submission Policy

UNSW has a standard late submission penalty of 5% per day, deducted from the maximum mark, capped at five days (120 hours) from the assessment deadline. After five days, a mark of zero will be awarded for the assignment. If you require an extension due to unforeseen circumstances, you must apply through the formal Special Consideration process on the UNSW website before the deadline. Tutors and lecturers cannot grant extensions directly.

Table 5: Notation Summary for IPPO and MAPPO Algorithms

Symbol	Description
N	Number of agents in the multi-agent environment
o_i	Local observation of agent i
$\mathbf{o} = \{o_1, \dots, o_N\}$	Set of all agents' local observations
s	Global state of the environment (MAPPO only)
s'	Next global state after executing joint action
a_i	Action selected by agent i
$\mathbf{a} = \{a_1, \dots, a_N\}$	Joint action of all agents
$P(a_i)$	Action probability distribution from policy π_{θ_i}
$P_{\text{new}}(a_i)$	Updated action probability under new policy parameters
$P_{\text{old}}(a_i)$	Behaviour policy probability used during data collection
r_i	Reward received by agent i
$\mathbf{r} = \{r_1, \dots, r_N\}$	Joint reward vector for all agents
$v_i = V_{\phi_i}(o_i)$	Value estimate from local critic (IPPO)
$v_i = V_{\phi_i}(s)$	Value estimate from centralized critic (MAPPO)
$v_{i,t}, v_{i,t+1}$	Value estimates at time t and $t + 1$
$\mathcal{B}_i$	Trajectory buffer for agent i
T_{steps}	Number of environment steps per rollout
$\delta_{i,t}$	Temporal-difference (TD) error for agent i at time t
$\hat{A}_{i,t}$	Generalized Advantage Estimate (GAE) for agent i
$R_{i,t}$	Return target for critic training
γ	Discount factor for future rewards
λ	GAE smoothing parameter
θ_i	Parameters of agent i 's actor network
ϕ_i	Parameters of agent i 's critic network
$\rho_t(\theta_i)$	PPO probability ratio for agent i at time t
L_{clip}	PPO clipped surrogate actor loss
L_{value}	Critic value regression loss
S_{entropy}	Policy entropy bonus encouraging exploration
L_{total}	Combined PPO loss (actor + critic + entropy)
c_1, c_2	Coefficients for critic loss and entropy bonus
K_{epochs}	Number of PPO optimization epochs per iteration

References

- [1] Stefano V Albrecht, Filippos Christianos, and Lukas Schäfer. *Multi-agent reinforcement learning: Foundations and modern approaches*. MIT Press, 2024.
- [2] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [3] Pablo Hernandez-Leal, Bilal Kartal, and Matthew E Taylor. A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 2019.
- [4] Jianhua Lin. Divergence measures based on the shannon entropy. *IEEE Transactions on Information theory*, 37(1):145–151, 1991.
- [5] John Schulman, Sergey Levine, Philipp Moritz, Michael I Jordan, and Pieter Abbeel. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1889–1897, 2015.
- [6] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [8] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, 1999.
- [9] Mark Towers, Ariel Kwiatkowski, John Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Kallinteris Andreas, Markus Krimmel, Arjun Kg, Rodrigo Perez-Vicente, et al. Gymnasium: A standard interface for reinforcement learning environments. *Advances in Neural Information Processing Systems*, 38, 2026.
- [10] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in neural information processing systems*, 35:24611–24624, 2022.